Docker Lab: Your First Image

Étudiant : Abdelbari Refak
Module : DevSecOps (S5)
Date : October 28, 2026

1 Introduction & Initialisation (Setup)

Ce laboratoire pratique aborde le cycle de vie complet de conteneurisation Docker, allant de l'exécution et de l'inspection de conteneurs existants jusqu'au durcissement de sécurité (exécution non-root, analyse Trivy) et la publication sur un registre privé distant (GitLab Container Registry).

1.1 Vérification de l'environnement

L'environnement local fonctionne sous Docker Desktop / WSL2 (Linux kernel standard). L'initialisation a été validée avec l'image officielle `hello-world`.
Question Q1. The image hello-world was not on your machine. Describe in your own words the steps Docker performed to show you this message:

This is what Docker explains about what it just did:

> Hello from Docker!
> This message shows that your installation appears to be working correctly.
>
> To generate this message, Docker took the following steps:
>
> 1. The Docker client contacted the Docker daemon.
> 2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64)
> 3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.

and this is what i think it does:

- **1_Checked your computer:** The system looked for the "hello-world" blueprint locally and realized it was missing.
- **2_Sent the request:** The command line tool you typed into passed your instructions to the main Docker engine operating in the background.
- **3_Downloaded the files:** Because the blueprint was missing, the background engine connected to an online library (Docker Hub) and downloaded the necessary "hello-world" files directly to your machine.
- **4_Ran the program:** The engine used those newly downloaded files to create an active, isolated environment (a container). It then ran the program inside that environment, which printed the welcome message to your screen.

2 Part 1 : Exécution et Inspection de Conteneurs Existants

2.1 Démarrage et observation du serveur Web Nginx

Un conteneur `nginx:1.27` a été instancié en arrière-plan (`-d`) sous le nom `web`, avec le port 80 du conteneur publié sur le port local 8080.

```
docker run -d --name web -p 8080:80 nginx:1.27
docker ps
docker logs web
docker exec -it web sh
ls /usr/share/nginx/html
exit
```

```
C:\Users\Admin\Documents\Downloads\S5\DevSecOps\lab_docker>docker ps
CONTAINER ID   IMAGE        COMMAND                  CREATED         STATUS         PORTS                                       NAMES
90a4b5df8e11   nginx:1.27   "/docker-entrypoint.…"   26 seconds ago   Up 25 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   web
```

Figure 1: Vérification du conteneur Nginx actif avec `docker ps` (Port 8080 → 80).

```
C:\Users\Admin\Documents\Downloads\S5\DevSecOps\lab_docker>docker logs web
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/28 08:40:04 [notice] 1#1: using the "epoll" event method
2026/10/28 08:40:04 [notice] 1#1: nginx/1.27.5
2026/10/28 08:40:04 [notice] 1#1: built by gcc 12.2.0 (Debian 12.2.0-14)
2026/10/28 08:40:04 [notice] 1#1: OS: Linux 6.18.33.2-microsoft-standard-WSL2
2026/10/28 08:40:04 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/10/28 08:40:04 [notice] 1#1: start worker processes
2026/10/28 08:40:04 [notice] 1#1: start worker process 29
2026/10/28 08:40:04 [notice] 1#1: start worker process 30
2026/10/28 08:40:04 [notice] 1#1: start worker process 31
2026/10/28 08:40:04 [notice] 1#1: start worker process 32
2026/10/28 08:40:04 [notice] 1#1: start worker process 33
2026/10/28 08:40:04 [notice] 1#1: start worker process 34
2026/10/28 08:40:04 [notice] 1#1: start worker process 35
2026/10/28 08:40:04 [notice] 1#1: start worker process 36
```

Figure 2: Consultation des journaux d'exécution du serveur Nginx via `docker logs web`.

```
C:\Users\Admin\Documents\Downloads\S5\DevSecOps\lab_docker>docker exec -it web sh
# ls
bin   dev                  docker-entrypoint.sh  home  lib64  mnt  proc  run   srv  tmp  var
boot  docker-entrypoint.d  etc                   lib   media  opt  root  sbin  sys  usr
# ls /usr/share/nginx/html
50x.html  index.html
#
```

Figure 3: Exploration interactive du système de fichiers interne via `docker exec -it web sh`.

2.2 Arrêt, suppression et cycle de vie

Les commandes `docker stop web` et `docker rm web` permettent de vérifier la persistance de l'image sur l'hôte même après la destruction de l'instance de conteneur.

```
C:\Users\Admin\Documents\Downloads\S5\DevSecOps\lab_docker>docker stop web
web
```

Figure 4: Arrêt du conteneur avec `docker stop web`.

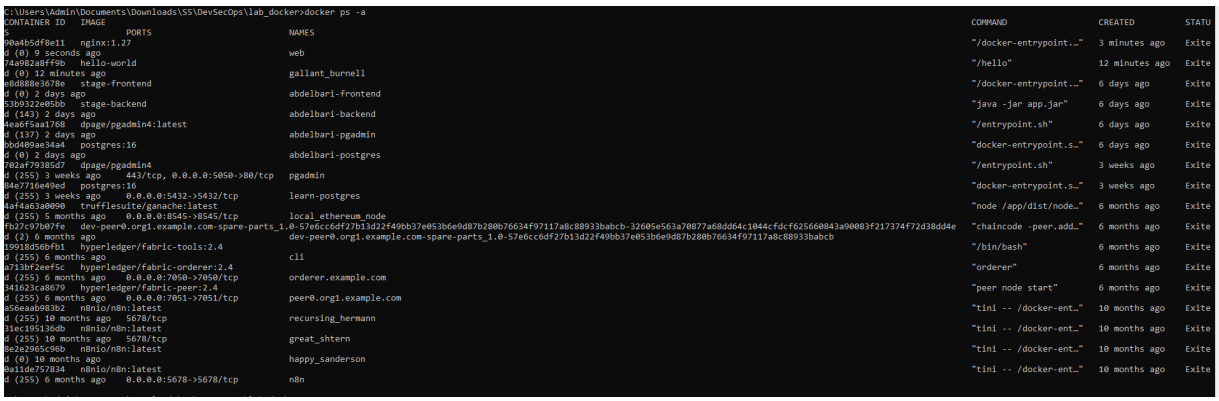


Figure 5: Inspection des conteneurs arrêtés avec `docker ps -a`.



Figure 6: Confirmation de la persistance de l'image `nginx:1.27` après `docker rm web`.

Question Q2. After docker rm web, the nginx image is still listed. What is the difference between an image and a container?

An image is a template, whereas a container is a running copy of that image.

- **Image:** This acts as a reusable blueprint where the application and its necessary environment travel together.
- **Container:** This is the active, running instance that is created from the image template.

Because they are two distinct concepts, stopping and removing a container only deletes the running copy of the app. The original image template is left intact on your machine and will still be listed until you specifically delete the image itself.

3 Part 2 : Création de la Première Image Docker

3.1 Préparation de l'application Python et test local

La micro-application de salutation est composée de `app.py` et d'un serveur HTTP natif `server.py` écoutant sur le port 8000.

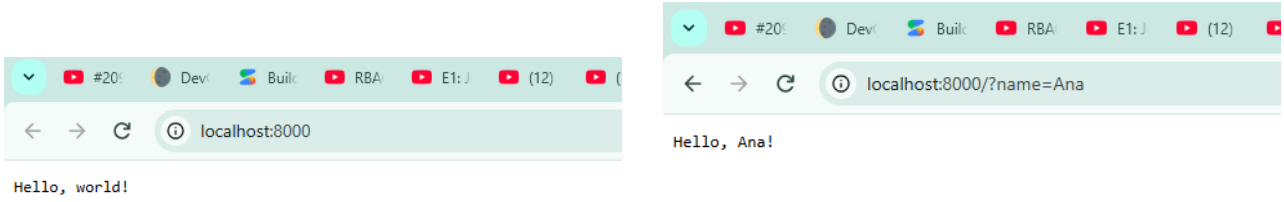


Figure 7: Test du serveur Python en local : route par défaut et paramètre `?name=Ana`.



Figure 8: Journaux d'écoute du serveur HTTP Python (`python3 server.py`).

3.2 Création du Dockerfile initial

Dans la branche Git `feature/11-dockerfile`, le Dockerfile v1.0 a été rédigé :

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py server.py ./
EXPOSE 8000
CMD ["python", "server.py"]
```

3.3 Construction, exécution et commit conventionnel

L'image a été construite sous le tag `greet:1.0`, puis lancée et testée avec `curl`.

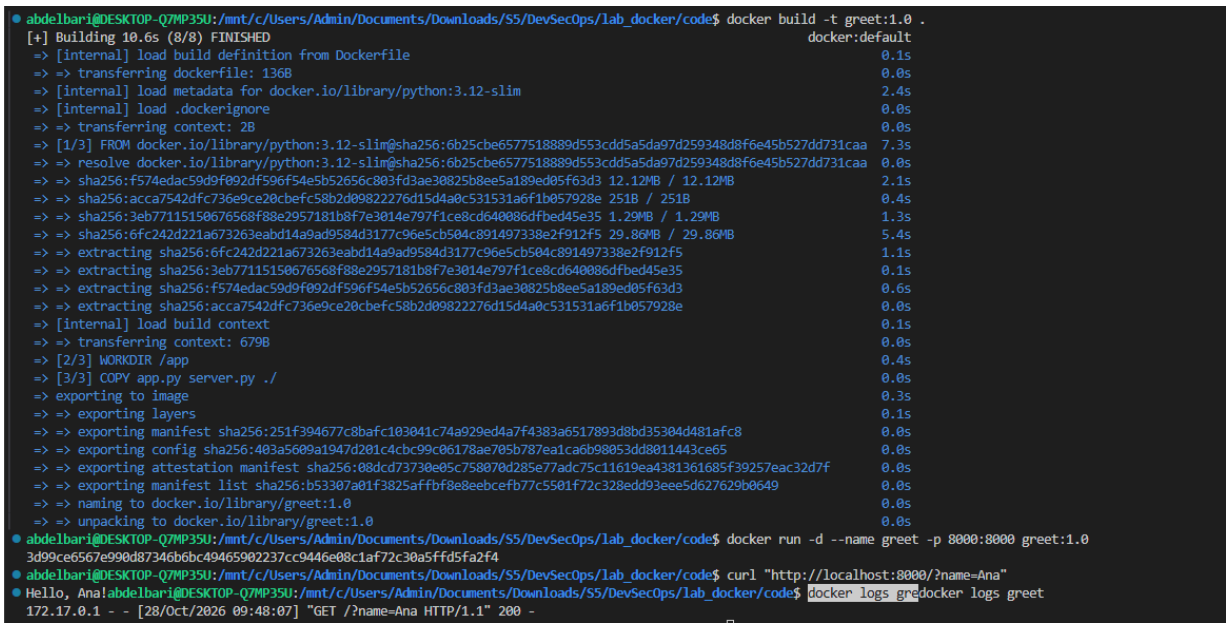


Figure 9: Construction de l'image `greet:1.0`, lancement du conteneur et test HTTP `curl`.

Le livrable Git a été consigné conformément aux conventions de commit :

```
git commit -m "feat(docker): add Dockerfile (#11)"
```

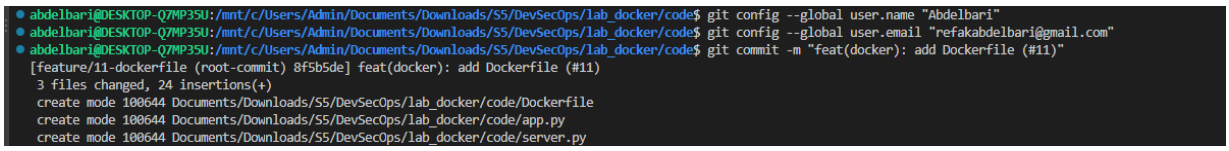


Figure 10: Validation du commit conventionnel pour l'Issue #11 sur la branche dédiée.

4 Part 3 : Compréhension des Couches, Cache et .dockerignore

4.1 Inspection des couches de l'image

La commande `docker history greet:1.0` affiche la structure en couches superposées créées par chaque instruction du Dockerfile.

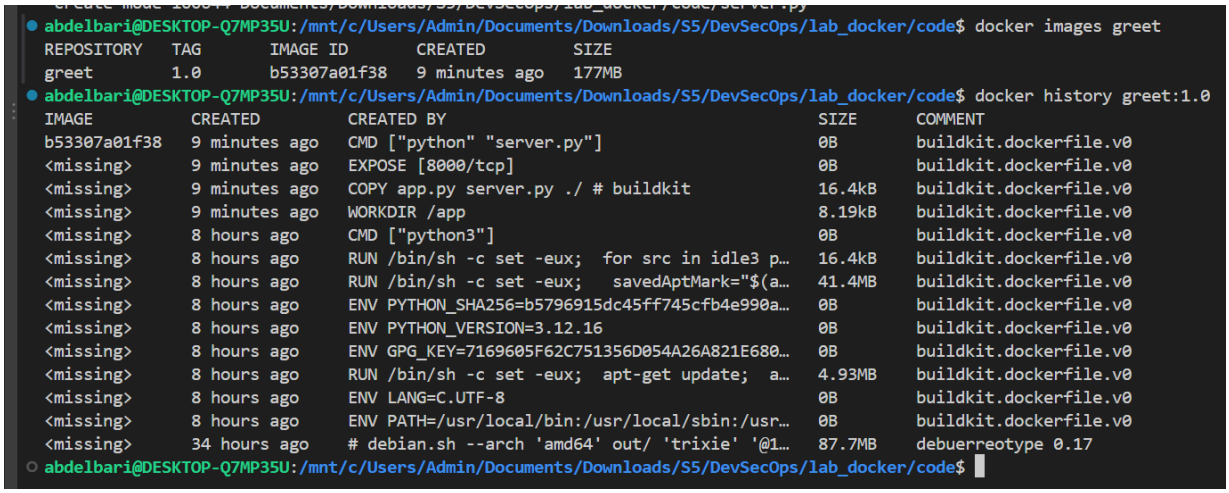


Figure 11: Analyse des couches de l'image `greet:1.0` avec `docker history`.

4.2 Mise en évidence du cache de build

Lors d'une modification du code source dans `app.py`, les couches antérieures (`WORKDIR /app`) restent en état `CACHED`, accélérant considérablement le temps de build.



Figure 12: Démonstration de la réutilisation des couches en cache (`CACHED`) lors de la reconstruction.

4.3 Sécurisation du contexte avec .dockerignore

Le fichier `.dockerignore` empêche l'injection accidentelle de secrets ou de fichiers temporaires dans le contexte de build :

```
.git
.venv
__pycache__/
.env
```

Question Q3. Why were the first layers reused when you rebuilt? What would happen if you copied your code before installing the dependencies of a larger project?

The first layers were reused because Docker saves the result of each instruction as a cache, and it recognized that the steps before the COPY command had not been altered. Instead of doing the same work twice, Docker just reused those saved layers and only rebuilt the parts from COPY onwards because that is where the files actually changed.

If I were to copy my application code before installing the dependencies for a larger project, any small change I make to my code would invalidate the cache for that step. Since Docker rebuilds everything that comes after a changed layer, it would have to completely re-download and reinstall all the project dependencies every single time I edit my code, defeating the purpose of ordering the Dockerfile to use the cache efficiently.

5 Part 4 : Sécurisation DevSecOps (Non-Root & Scan Trivy)

5.1 Détection de l'exécution avec privilèges Root

L'exécution de la commande `whoami` à l'intérieur du conteneur en cours d'exécution révèle que l'application tourne par défaut sous le compte super-utilisateur `root`.

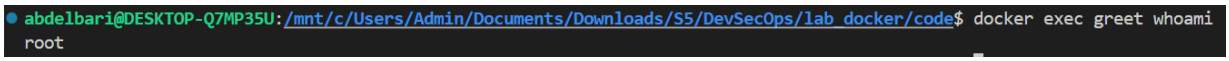


Figure 13: Constat de la vulnérabilité : l'application s'exécute avec l'utilisateur `root`.

5.2 Audit automatisé avec Trivy

Le scanner de sécurité Trivy (`aquasec/trivy:0.75.0`) a été exécuté en mode conteneur pour analyser le Dockerfile. Il a remonté une anomalie de sévérité **HIGH** (règle `DS-0002`) :

> *DS-0002 (HIGH): specify at least 1 USER command in Dockerfile with non-root user as argument. Running containers with root user can lead to a container escape situation.*

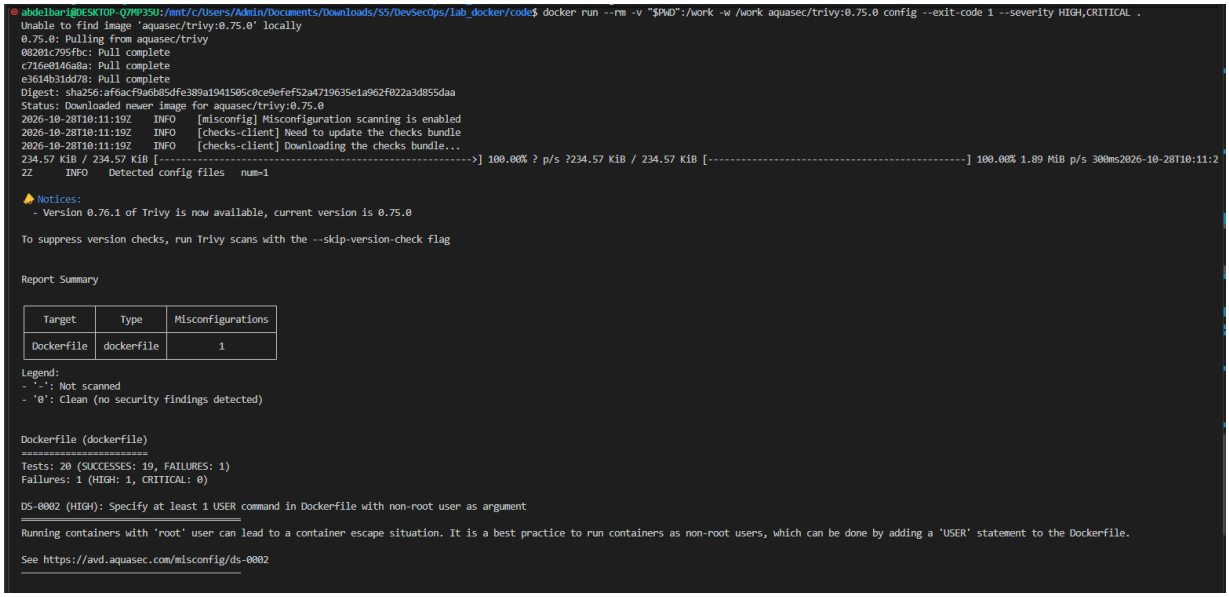


Figure 14: Scan de vulnérabilité Trivy signalant l'alerte HIGH (absence d'instruction USER non-root).

5.3 Remédiation et Durcissement du Dockerfile

Sur une nouvelle branche Git `fix/12-non-root`, le Dockerfile a été modifié pour créer et utiliser un utilisateur non-privilégié `appuser` :

```
FROM python:3.12-slim
RUN useradd --create-home appuser
USER appuser
WORKDIR /home/appuser
COPY app.py server.py ./
EXPOSE 8000
CMD ["python", "server.py"]
```

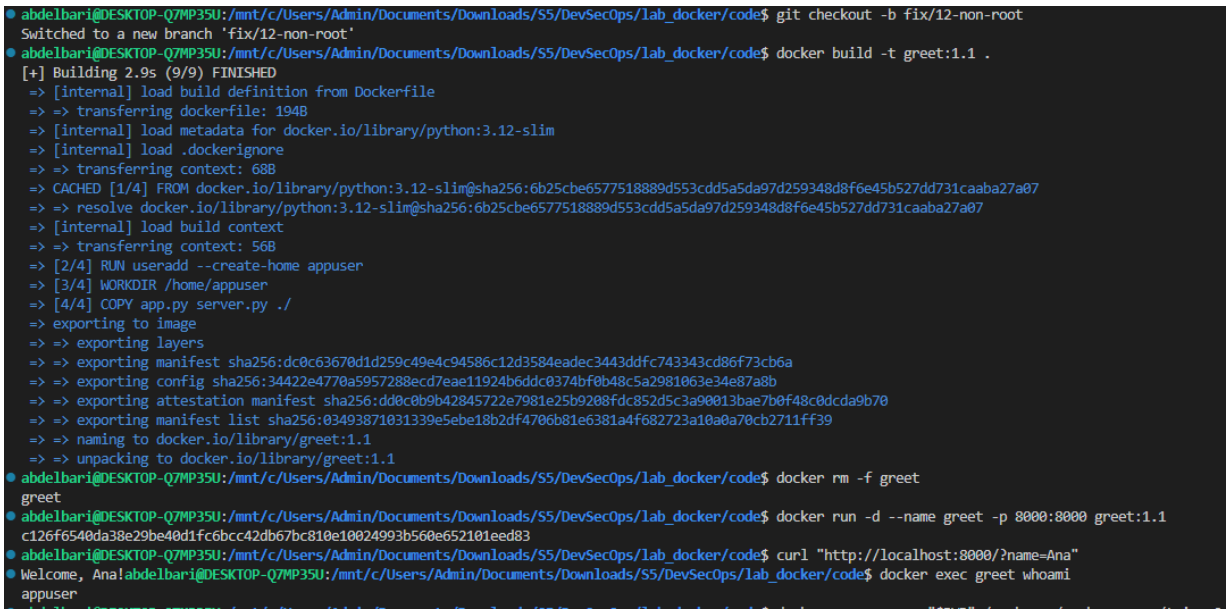


Figure 15: Reconstruction sous le tag `greet:1.1` et confirmation avec `whoami` de l'identité `appuser`.

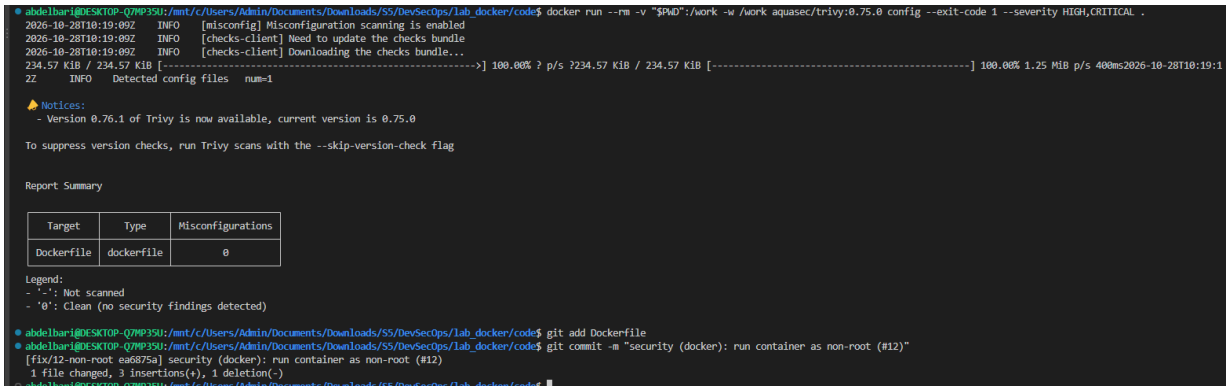


Figure 16: Nouveau scan Trivy propre (Clean / 0 vulnérabilité) et commit conventionnel (#12).

Question Q4. Why is running a container as root dangerous even though the container is isolated?

Running a container as root is dangerous because if an attacker exploits a vulnerability in your app, they start with maximum rights inside the container. Even though containers are isolated, having root access makes a potential attack much worse, as it gives the attacker the tools and permissions they need to attempt a "container breakout" and potentially compromise the host machine.

6 Part 5 : Publication sur le Registre Privé GitLab

6.1 Authentification sécurisée avec Personal Access Token

La connexion au registre Docker de GitLab s'effectue de manière sécurisée en injectant le Personal Access Token via l'entrée standard (`-password-stdin`) :

```
echo "<your-token>" | docker login registry.gitlab.com -u abdelbarirefak --
    password-stdin
```



Figure 17: Authentification réussie au registre GitLab et marquage (tag) de l'image `1.1`.

6.2 Tagging et Push vers GitLab Container Registry

L'image durcie a été étiquetée avec précision puis téléversée sur le registre de projet :

```
docker tag greet:1.1 registry.gitlab.com/abdelbari-group/devsecops-lab/greet
    :1.1
docker push registry.gitlab.com/abdelbari-group/devsecops-lab/greet:1.1
```

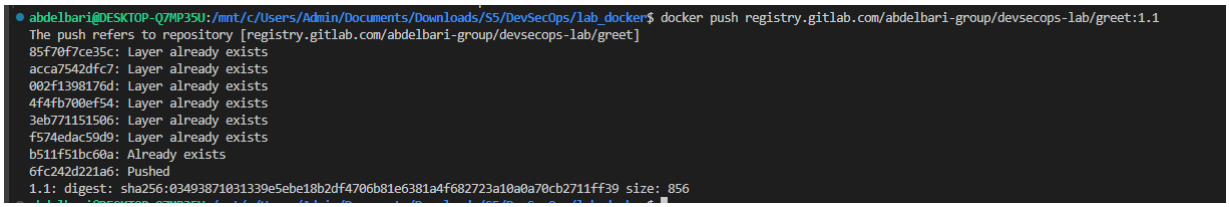


Figure 18: Publication des couches de l'image vers le dépôt distant GitLab.

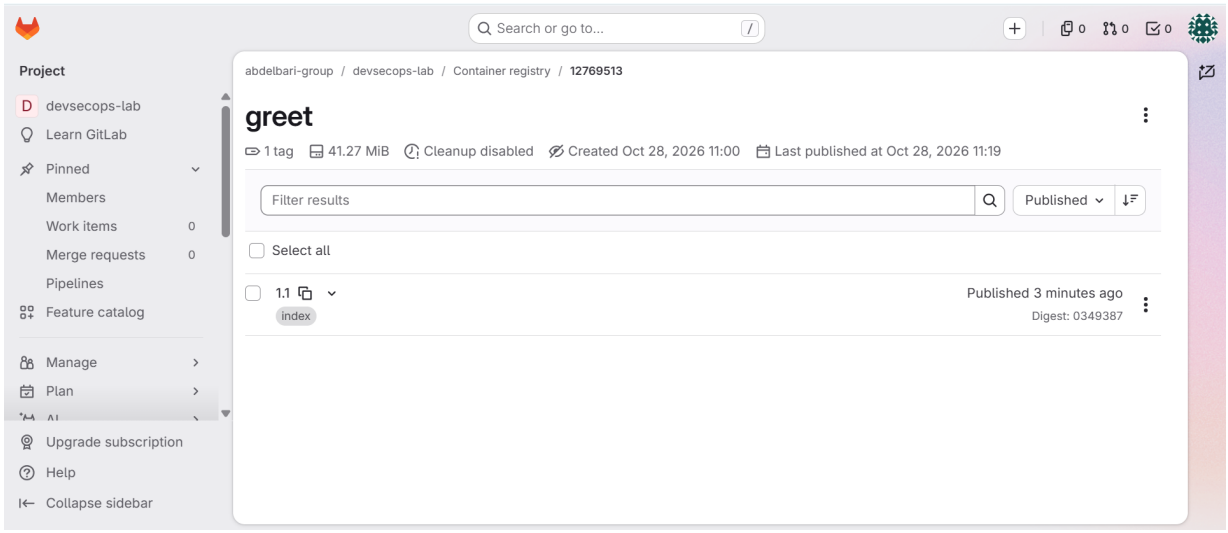


Figure 19: Interface GitLab Container Registry confirmant la disponibilité de l'image `greet:1.1`.

Question Q5. We pushed the tag 1.1 and never used latest. Why is a precise version tag better?

If I just rely on the latest tag, it acts like a moving target because it can be overwritten anytime someone pushes a new update to the registry. By using a precise tag like 1.1, I know exactly which version of the code and environment I am downloading, ensuring the application runs predictably every time and doesn't suddenly break due to unexpected background changes.

7 Part 6 : Nettoyage et Revue des Bonnes Pratiques

7.1 Nettoyage des conteneurs et des images orphelines

Après publication, les ressources locales ont été supprimées et nettoyées avec `docker image prune`.

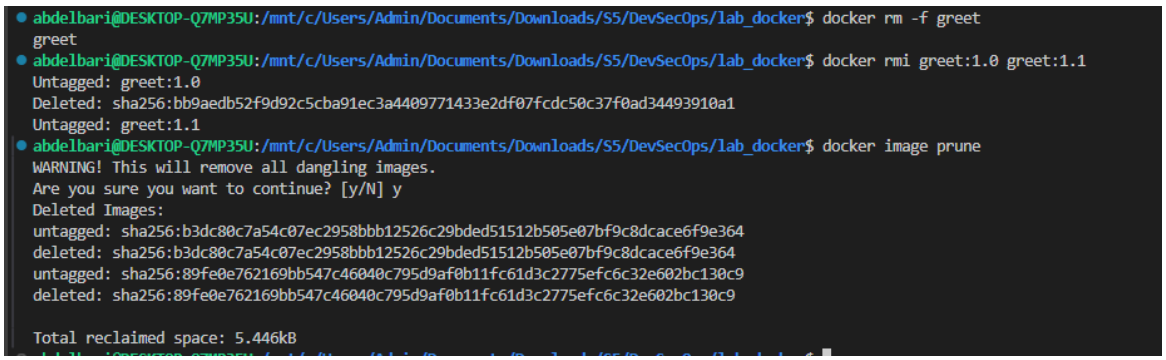


Figure 20: Nettoyage complet des conteneurs locaux et récupération d'espace disque.

7.2 Dockerfile Checklist Review (Synthèse Finale)

Revue formelle de conformité :

My Dockerfile successfully meets all the security and best-practice requirements outlined in the final checklist. First, the base image version is strictly pinned by starting from `python:3.12-slim`. Second, the container safely runs as a non-root user because the configuration creates and switches to `appuser`. Additionally, a `.dockerignore` file is in place to guarantee that no local secrets or unnecessary junk files are accidentally copied into the image during the build process. Finally, as a result of these configurations, the Trivy security scan passes without finding any root-access vulnerabilities.